

Message Queues and Asynchronous Processing

Introduction

Not every piece of work a system needs to do belongs inside the request-response cycle of an HTTP call. Sending a confirmation email, generating a large report, resizing an uploaded image, or notifying a downstream system of an event are all tasks that a client should not have to wait on before receiving a response. Message queues exist to decouple the moment work is requested from the moment work is actually performed, by placing a durable, ordered record of "work to do" onto a queue that one or more worker processes consume independently of the original request. This decoupling is what allows a web server to respond in milliseconds while a report that takes thirty seconds to generate happens safely in the background.

Producers, Queues, and Consumers

Every queueing system shares the same three roles. A producer is the part of the system that creates a message and publishes it onto a queue, typically the web application handling the original request. The queue itself is a durable, ordered (or partially ordered) store of pending messages, usually implemented by dedicated infrastructure such as RabbitMQ, Amazon SQS, or Kafka rather than being built by hand. A consumer, often called a worker, pulls messages off the queue and performs the actual work, such as calling an email provider or writing to a database. Because producers and consumers never communicate directly, either side can be scaled, restarted, or temporarily unavailable without the other side needing to know or care, as long as the queue itself remains available.

At-Least-Once vs Exactly-Once Delivery

Most production queueing systems guarantee at-least-once delivery, not exactly-once delivery, and this distinction has real consequences for how workers must be written. At-least-once means a message is guaranteed to be delivered, but under certain failure conditions, such as a worker crashing after processing a message but before acknowledging it, the same message may be redelivered and processed again. Because of this, worker logic should be idempotent wherever possible, meaning that processing the same message twice produces the same end result as processing it once, for example by checking whether an order has already been marked as shipped before shipping it again. True exactly-once delivery is difficult to guarantee across distributed systems and is usually approximated rather than strictly achieved, so idempotency in the consumer is the more reliable

engineering answer than relying on the queue to prevent duplicates outright.

Acknowledgment and Visibility Timeouts

A message is not considered fully processed until the consumer explicitly acknowledges it back to the queue. If a worker picks up a message and crashes before acknowledging it, a well-designed queue will eventually make that message available again for another worker to pick up, rather than losing it silently. This is typically implemented through a visibility timeout: once a worker receives a message, the queue hides it from other consumers for a set duration, assuming the worker is actively processing it. If the worker acknowledges the message before the timeout expires, it is removed from the queue permanently. If the timeout expires first, with no acknowledgment received, the message becomes visible again and can be picked up by another worker. Setting this timeout too short causes messages to be reprocessed by multiple workers simultaneously; setting it too long delays recovery when a worker genuinely has crashed.

Dead Letter Queues

Some messages fail no matter how many times they are retried, for example a malformed message that triggers the same parsing error on every attempt. Without a safeguard, such a message could be retried indefinitely, consuming worker capacity without ever succeeding. A dead letter queue solves this by automatically moving a message to a separate queue after it has failed a configured number of times, removing it from the main processing flow so healthy messages are not blocked behind it. The dead letter queue then becomes a place engineers can inspect manually, understand why a batch of messages kept failing, and either fix the underlying bug and reprocess them, or discard them if they turn out to be genuinely invalid. Systems without a dead letter queue often discover failure patterns only when a queue backs up unexpectedly, which is a much harder starting point for debugging.

Choosing a Queue Technology

The right queueing technology depends heavily on the shape of the workload rather than raw throughput numbers alone. RabbitMQ is a strong default for traditional task queues where flexible routing rules (fanout, topic, direct exchanges) matter more than extreme throughput, and where message-level acknowledgment semantics need to be precise. Kafka is built around high-throughput, append-only log storage and is the better fit when messages need to be replayed, consumed by multiple independent consumer groups, or retained for long periods rather than deleted immediately after processing. Managed services such as Amazon SQS remove operational overhead entirely at the cost of some flexibility, and are

often the pragmatic choice for teams that do not want to operate queueing infrastructure themselves. None of these is universally "best"; the deciding factors are replay requirements, ordering needs, expected throughput, and how much operational burden the team is willing to take on.

Ordering Guarantees

Whether a queue preserves the order in which messages were produced varies significantly between systems and configurations. Kafka, for instance, guarantees ordering only within a single partition, so messages that must be processed in strict order relative to each other need to be routed to the same partition, commonly by using a consistent key such as a user ID or order ID. Standard queues such as SQS offer no ordering guarantee by default, and only their FIFO variant preserves order, typically at the cost of reduced throughput compared to the standard mode. A frequent design mistake is assuming a queue preserves global ordering when it does not, which can cause subtle bugs such as an "item updated" event being processed before the corresponding "item created" event simply because they landed on different partitions or were retried at different times.

Backpressure and Consumer Scaling

When producers generate messages faster than consumers can process them, the queue begins to grow, a condition usually visible as increasing queue depth or consumer lag. The correct response is rarely to simply let the queue grow unbounded; instead, systems typically autoscale the number of consumer workers based on queue depth, similar to how web servers autoscale based on request volume. It is also common to apply backpressure further upstream, for example by having the producer itself slow down or reject new work once the queue passes a certain depth, rather than allowing unbounded queue growth to eventually exhaust storage or dramatically increase end-to-end latency for every message waiting in line.

Summary and Practical Checklist

Message queues are the standard tool for moving work out of the synchronous request path, and the guarantees they offer are more nuanced than "the message will be delivered." Because most systems provide at-least-once delivery rather than exactly-once, consumer logic should be written to be idempotent by default. Visibility timeouts need to be tuned against how long processing actually takes, not left at a default value, since too short a timeout causes duplicate processing and too long a timeout delays recovery from real worker failures. Dead letter queues should be treated as a required safety net rather than an

optional extra, since without one a single malformed message can silently block or endlessly consume capacity within a queue. Finally, ordering guarantees should always be verified against the specific queue technology in use rather than assumed, since the default behavior of many popular systems is to provide no strict ordering at all.